# Presentazione progetto: CatcheM_CV

Riccardo Canonico (mat. 335359)

Lorenzo Mastrini (mat. 336478)

Ingegneria del Software, Prof. Alfredo Milani

# OBIETTIVO DEL PROGETTO

Sviluppo di un gioco a livelli in realtà aumentata utilizzando OpenCV, libreria software nell'ambito di Computer Vision.

# DESCRIZIONE GENERALE



# IL GIOCO

Durante il gioco l’obiettivo principale del giocatore è guidare delle palline all’interno di un target disegnato su schermo.

Per vincere si dovrà raggiungere un certo punteggio entro un tempo limite.

Le palline possono essere di 2 tipi:

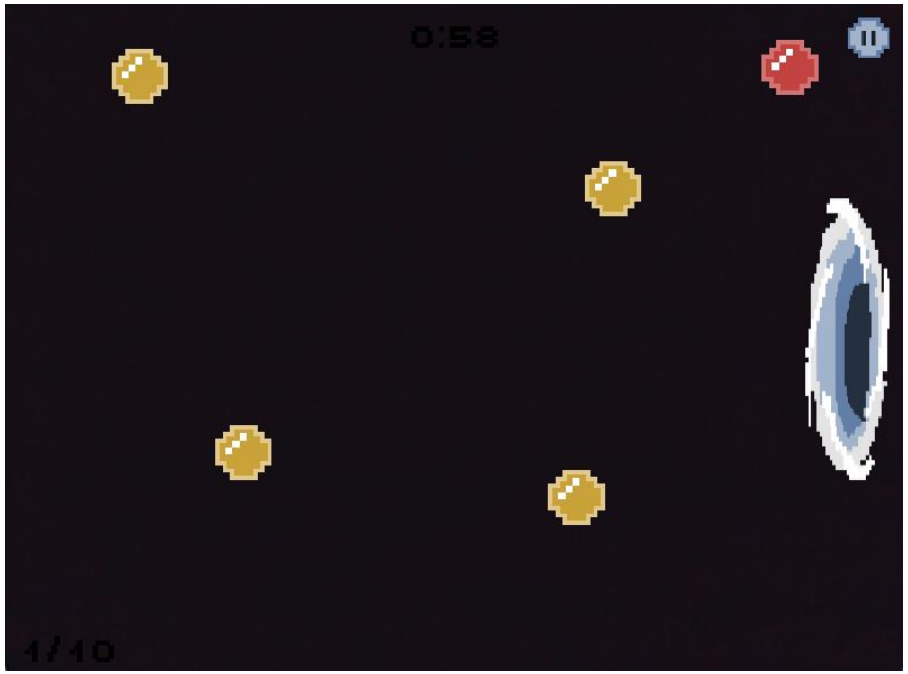
- Gialle: palline che aumentano il punteggio
- Rosse: palline che diminuiscono il punteggio; il valore sottratto dipenderà dal livello selezionato

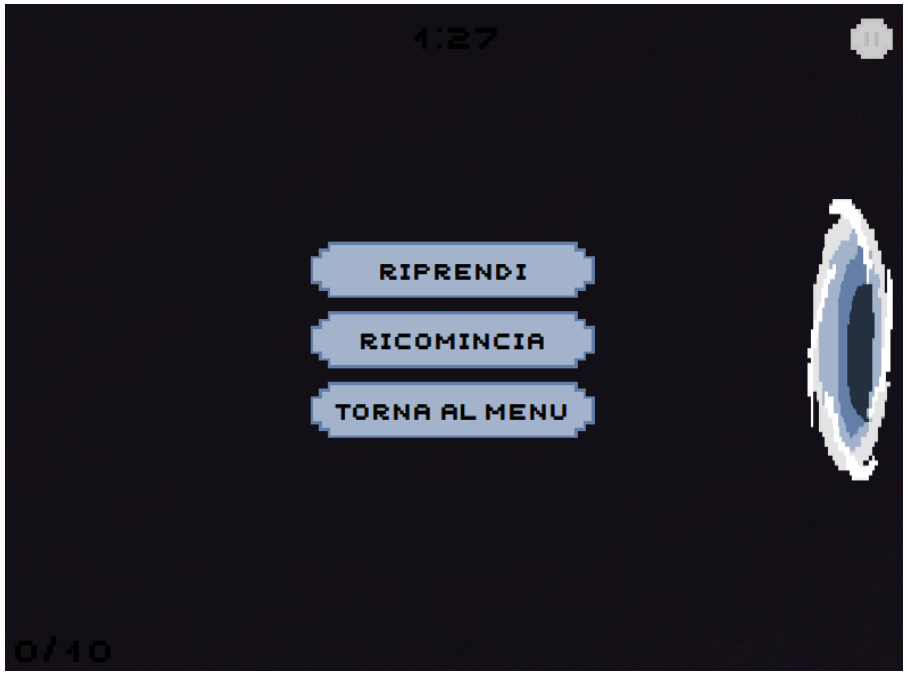
# MENU PRINCIPALE




Questa è la schermata che verrà presentata al giocatore all'avvio dell'applicazione, dove potrà scegliere il livello o uscire dal gioco

# LIVELLO

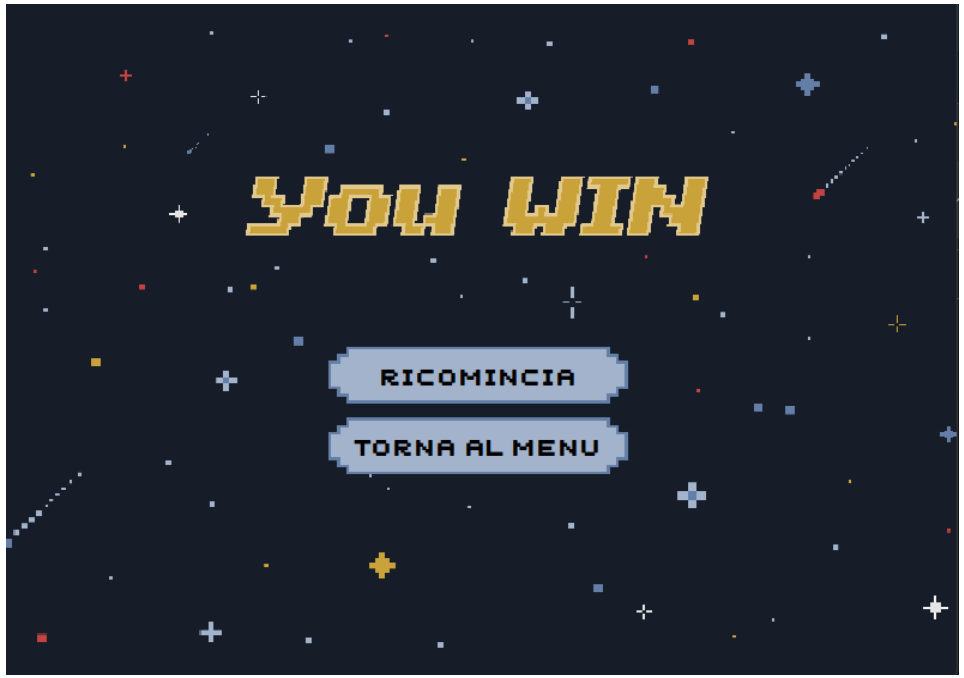


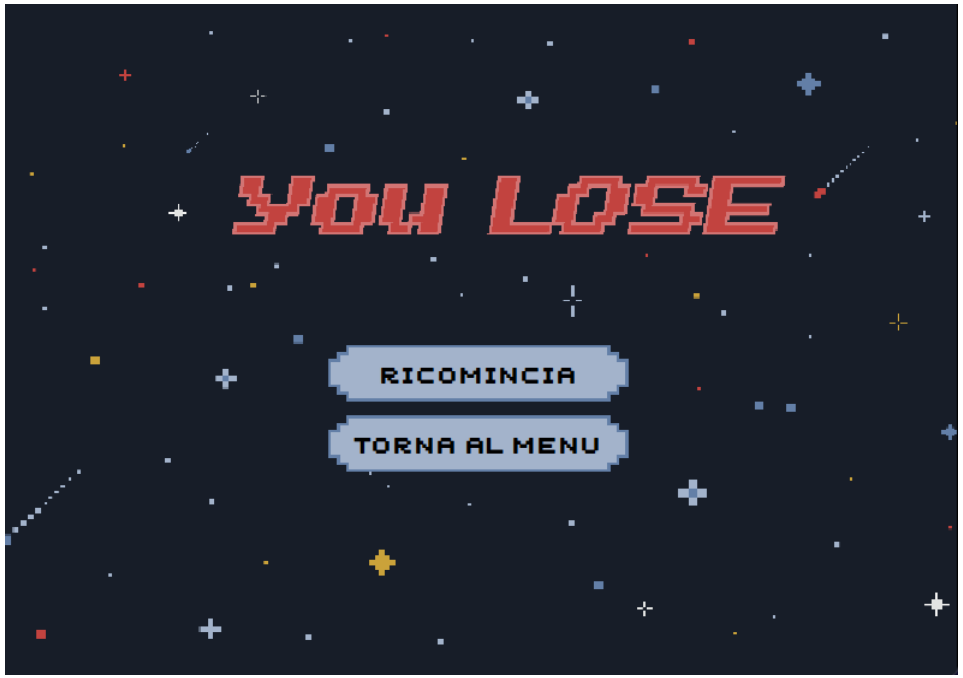




Questa è la schermata dedicata al livello (in questo caso intermedio o avanzato data la presenza della pallina rossa), dove il giocatore potrà guidare le palline verso il target (il portale) e mettere in pausa il gioco.

Durante la pausa il giocatore potrà riprendere la partita, ricominciare il livello o tornare al menu principale.

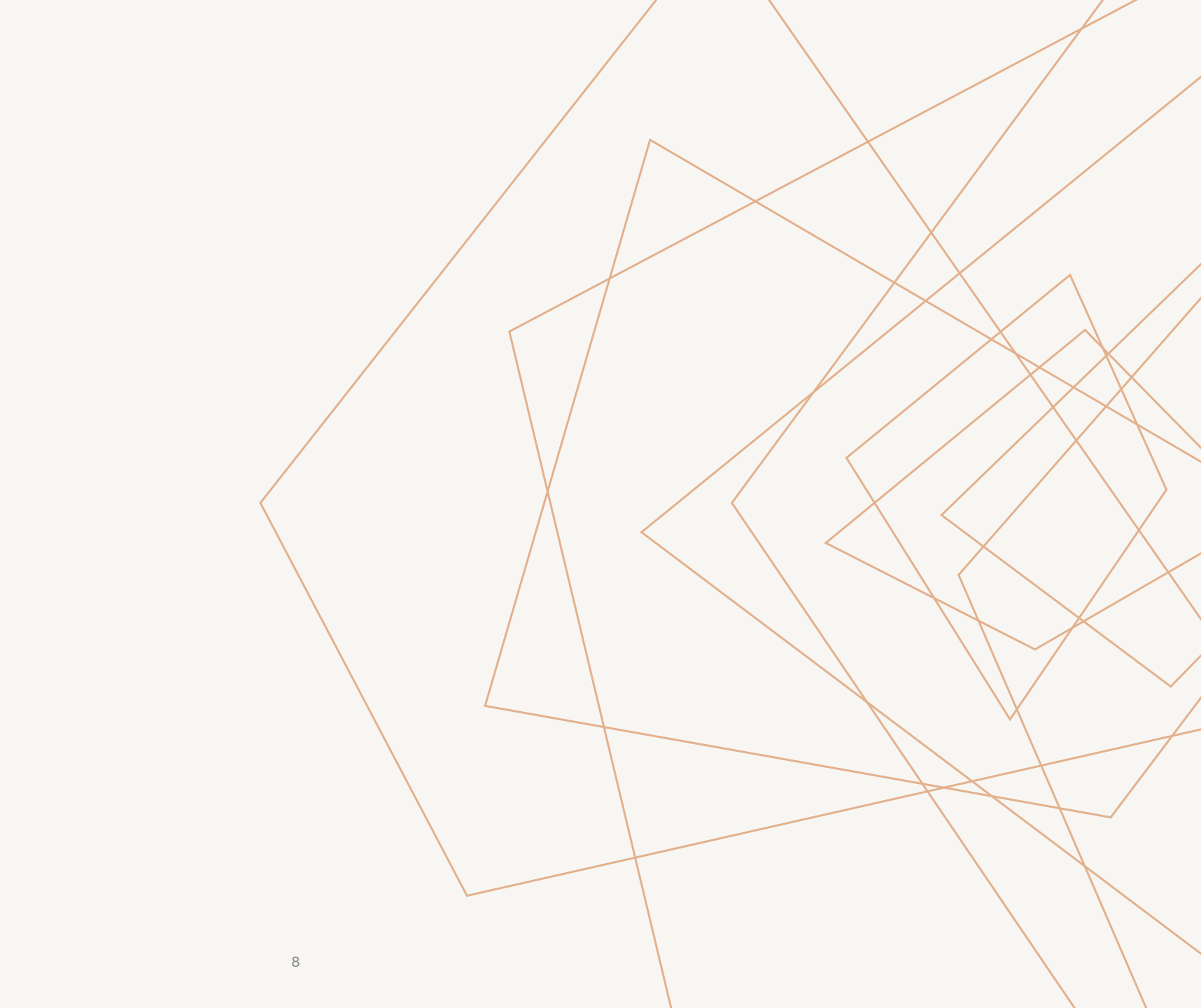
# PARTITA




Questa è la schermata che viene presentata a fine livello, a seconda se il giocatore ha vinto o perso.
Anche in questo caso sarà possibile ricominciare o tornare al menu.

# DEMO



# ARCHITETTURA DEL SISTEMA



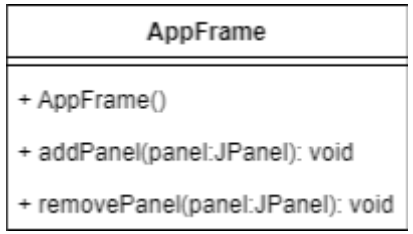
# CLASSI COINVOLTE

- Main
- Level
- Ball
- Target
- OpticalFlow
- ImgProcessor

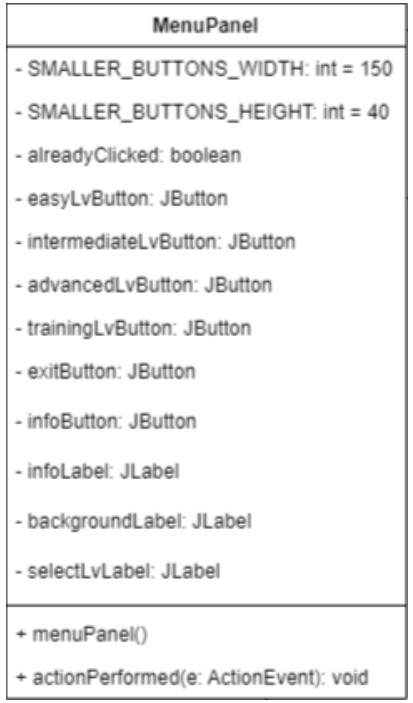
- Frame
- MenuPanel
- GamePanel
- PausePanel
- EndGamePanel

# PANNELLI GRAFICI

AppFrame rappresenta la finestra principale del gioco, sulla quale vengono aggiunti e tolti i vari pannelli che fanno da interfaccia grafica all'applicazione.
Per fare ciò vengono usati i metodi 'addPanel()' e 'removePanel()'.




I pannelli avranno tutti come attributi gli elementi grafici che li compongono e il metodo 'actionPerformed()' che definisce il comportamento dei vari bottoni.

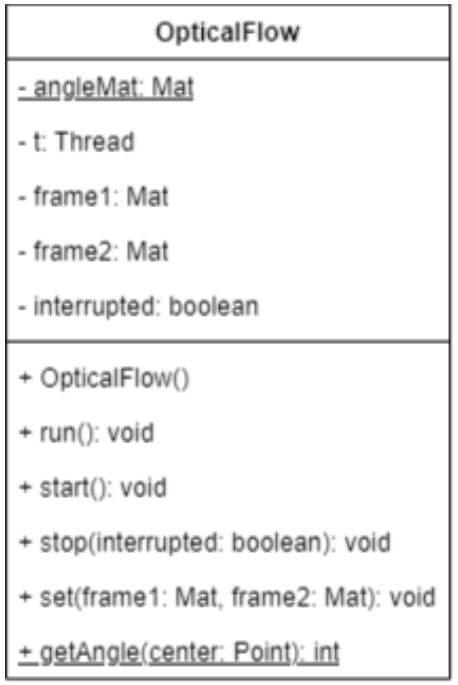



```java
@Override
public void actionPerformed(ActionEvent e) {
    //level 1
    if (e.getSource() == easyLvButton) {
        easyLvButton.setEnabled(false);
        Level baseLevel = new Level(1); //instantiate the level at base difficulty
        baseLevel.start();
    }
```

# COMPUTER VISION

Le classi trattate in seguito sono coinvolte nelle operazioni di visione artificiale e analisi dell'immagine.

OpticalFlow implementa l'omonimo algoritmo, che attraverso varie operazioni matematiche e confrontando due frame consecutivi rileva movimenti del giocatore.
Oltre a ciò, in base a quello che viene calcolato stimerà l'angolo del movimento con il metodo 'getAngle()'. Questo servirà a direzionare le palline.



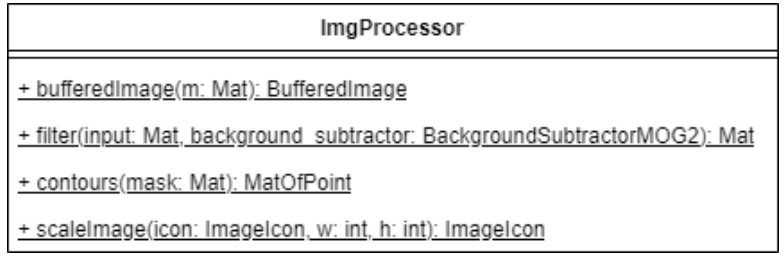

```java
public static int getAngle(Point center) {
    //this code is for limiting the zone of calculation of optical flow, based on the center of the ball
    int elements = 0; //number of sub-matrix pixels considered in the calculation of the mean of the angles
    double angle_sum = 0;

    //this "builds" a square around the ball to detect only the movements of the pixels inside it
    for (int j = (int) center.x - 1; j < (int) center.x + 1; j++) {
        for (int i = (int) center.y - 1; i < (int) center.y + 1; i++) {
            if (center.x - 20 >= 0 && center.x + 20 <= Level.maxWidth && center.y - 20 >= 0 && center.y <= Level.maxHeight) {
                if (!angleMat.empty()) {
                    angle_sum += angleMat.get(i, j)[0];
                    elements++;
                }
            }
        }
    }
    int angle_mean = 0;
    angle_mean = (int) (angle_sum / elements) % 360; //all the angles are between 0 degrees and 360 degrees

    return angle_mean;
}
```

# COMPUTER VISION

ImgProcessor si occupa principalmente di effettuare specifiche operazioni sull'immagine.
Un esempio è 'Contours()' che si occupa di rilevare i contorni della mano del giocatore, utilizzati poi per capire quando quest'ultima entra in collisione con le palline, per poi farle rimbalzare.




```java
public static MatOfPoint contours(Mat mask) {
    //detect all contours
    List<MatOfPoint> contours = new ArrayList<>();
    Mat hierarchy = new Mat(); //not used
    Imgproc.findContours(mask, contours, hierarchy, Imgproc.RETR_EXTERNAL, Imgproc.CHAIN_APPROX_SIMPLE);

    //find biggest contours
    int maxContourArea = -1000;
    MatOfPoint biggestContours = new MatOfPoint();
    for (int i = 0; i < contours.size(); i++) {
        if (Imgproc.contourArea(contours.get(i)) > maxContourArea) {
            maxContourArea = (int) Imgproc.contourArea(contours.get(i));
            //biggestContourIndex = i;
            biggestContours = contours.get(i); //extract from all contour list only biggest contours (hand)
        }
    }

    return biggestContours;
}
```

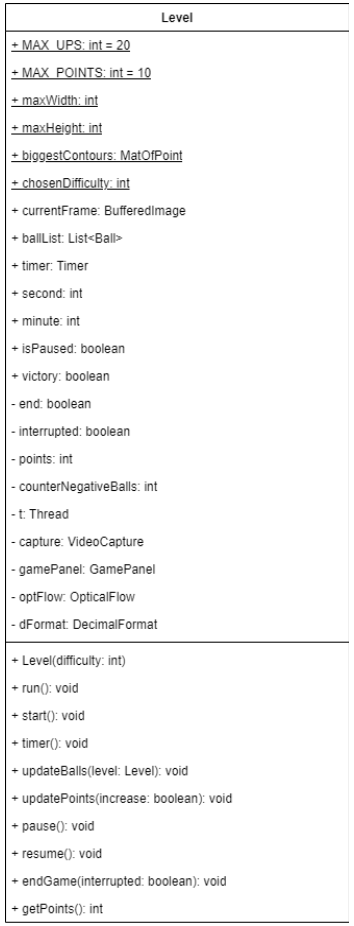
# LIVELLO GENERALE

Di seguito sono trattate le classi che implementano la logica dei vari elementi del gioco.

Level è la classe principale tra quelle che descriveremo, in quanto implementa la logica di un qualsiasi livello dell'applicazione e gestisce gli oggetti delle classi che si occupano di Computer Vision.

Inoltre coordina le palline insieme alla classe GamePanel, tiene traccia del punteggio, del timer, ed effettua i vari controlli legati alla vittoria o sconfitta.

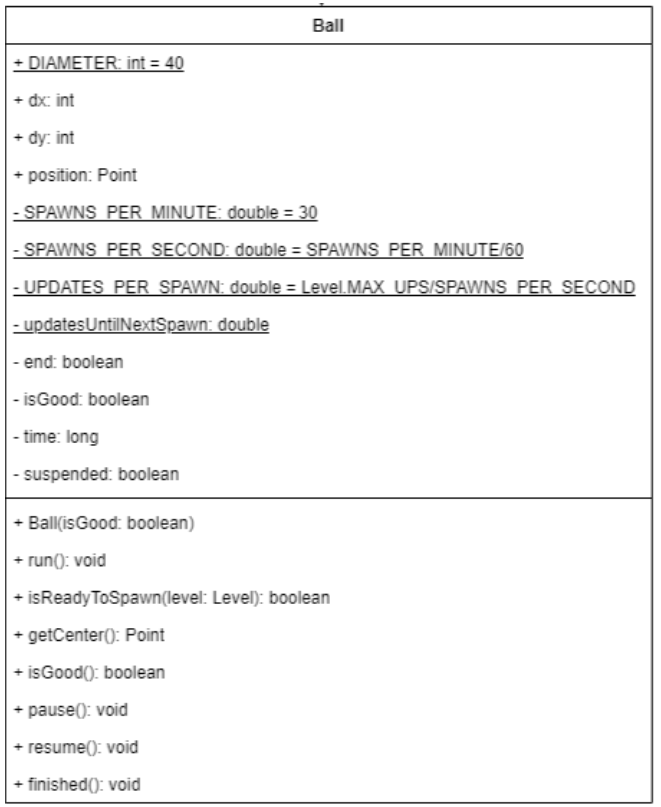
Questo viene fatto all'interno del ciclo principale nel metodo 'run()', dopo una fase di inizializzazione degli attributi.



# LIVELLO GENERALE

Ball rappresenta la singola pallina con cui il giocatore interagisce.

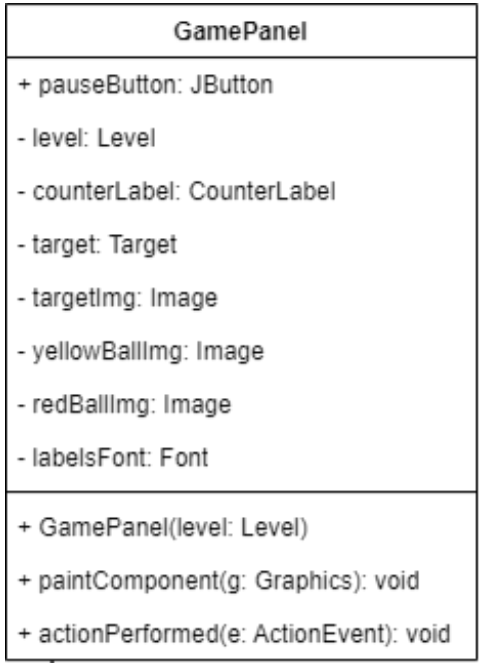
Insieme alla classe OpticalFlow gestisce, all'interno del metodo 'run()', il proprio movimento. Inoltre con il metodo 'isReadyToSpawn()' e vari attributi della classe, viene controllato il numero di palline contemporaneamente disegnate a schermo.




```
if (angle >= 337 || angle < 23) { //0 degrees
    dx = 5;
    dy = 0;
} else if (angle >= 23 && angle < 67) { //45 degrees
    dx = 5;
    dy = 5;
} else if (angle >= 67 && angle < 112) { //90 degrees
    dx = 0;
    dy = 5;
} else if (angle >= 112 && angle < 157) { //135 degrees
    dx = -5;
    dy = 5;
} else if (angle >= 157 && angle < 202) { //180 degrees
    dx = -5;
    dy = 0;
} else if (angle >= 202 && angle < 247) { //225 degrees
    dx = -5;
    dy = -5;
} else if (angle >= 247 && angle < 292) { //270 degrees
    dx = 0;
    dy = -5;
} else if (angle >= 292 && angle < 337) { //315
    dx = 5;
    dy = -5;
}
```

# LIVELLO GENERALE

GamePanel è la classe che, oltre ad implementare ‘actionPerformed()’ come gli altri pannelli per il bottone di pausa, disegna i principali elementi grafici del gioco (palline, target, timer, punteggio) attraverso ‘paintComponent()’, la cui logica poi è gestita da Level.



# GRAZIE PER L’ATTENZIONE